Problem Statement Proposal

DBMS Experiential Learning - Level 3 Project

Municipal Civic Grievance Redressal System with Semantic Duplicate Detection and SLA-Based Auto-Escalation

1. Problem Statement

Citizens report civic issues - potholes, garbage accumulation, water leakage, broken streetlights - through a web platform. The system automatically detects when a new report is semantically similar to an existing one, even when the wording is completely different, and flags it as a likely duplicate. Every complaint carries a resolution SLA; if a department fails to act within that window, the system automatically escalates the complaint to the next authority level and notifies the relevant stakeholders. This mirrors real systems such as India's CPGRAMS and various municipal grievance portals, but is rarely attempted as a student DBMS project - most teams default to library, hotel, hospital, or banking systems, and even complaint-tracker variants typically stop at plain CRUD.

2. Why This Problem Statement

- **Goes beyond CRUD:** duplicate-matching and SLA-triggered escalation are genuine workflow logic, not just insert/update/delete operations.
- **Uncommon domain:** rarely used in DBMS coursework, satisfying the uniqueness and originality expectation.
- **Socially meaningful:** addresses a real civic administration problem with a clear real-world analogue.

3. Database Paradigms - Option 2 (SQL + Vector Database)

- **SQL (PostgreSQL):** Users, Wards, Departments, Complaints, StatusHistory, Escalations - a clearly relational, foreign-key-heavy structure.
- **Vector Database (pgvector or Qdrant):** complaint text (title and description) is embedded and used for semantic duplicate detection.

This choice satisfies the mandatory minimum of two database paradigms and simultaneously fulfills the rubric's recommendation to use a vector database for unstructured, text-heavy data, since free-text complaint descriptions are exactly that kind of data.

4. Database Design Readiness

The entity list above supports a properly normalized relational design: complaints link to wards, departments, and assigned staff; status changes are tracked historically; escalations form a traceable chain. This gives sufficient depth for a complete ER diagram and relational schema (not included here, per request, but confirmed feasible).

5. SQL Implementation Depth (DDL / DML)

- **DDL:** CREATE / ALTER / DROP across the schema; primary and foreign keys; constraints; indexes on ward_id, department_id, status, and created_at.
- **DML:** INSERT / UPDATE / DELETE / SELECT throughout; JOINs across complaint-ward-department-staff; GROUP BY / HAVING (e.g. departments with more than five complaints past SLA); aggregate functions (average resolution time); subqueries (complaints delayed beyond their department's average).

6. Advanced Database Features (minimum 4 required - 7 available here)

- **Triggers:** SLA-breach auto-escalation and notification generation; ward-level complaint counters.
- **Stored procedures / functions:** auto-assignment of new complaints to the correct department or staff member; SLA-compliance rate computation.
- **Views:** public dashboard of open and resolved complaint counts per ward and department.
- **Transactions:** status update, escalation log entry, and notification insert committed atomically.
- **Indexing:** on ward, department, status, and date fields for dashboard query performance.
- **Full-text search:** keyword search for citizens checking whether an issue has already been reported.
- **Vector similarity search:** semantic duplicate detection that catches paraphrased reports full-text search would miss.

7. Vector Database Documentation

Complaint title and description text is converted to embeddings using a sentence-embedding model; vectors are stored in a pgvector column; cosine similarity is used to retrieve the top-k nearest neighbors; any result above a defined similarity threshold is flagged as a likely duplicate. This satisfies the documentation requirements around embedding approach, storage, similarity metric, and retrieval strategy.

8. Roles and Authentication (minimum 2 required - 3 used here)

- **Citizen:** submit, track, and upvote complaints.
- **Department Staff:** manage an assigned complaint queue.
- **Ward / Municipal Admin:** view cross-department analytics, reassign complaints, oversee escalations.

These roles reflect genuinely different permissions required by the problem domain, rather than being added purely for compliance.

9. Data Validation and Integrity

Complaint status is constrained to a valid state machine (Open -> Assigned -> In Progress -> Resolved -> Closed/Reopened), enforced via a CHECK constraint or trigger. Every complaint is foreign-key-bound to a valid ward and department, and contact information is format-validated at the database level.

10. Team Split

One member can own the database layer - schema, triggers, procedures, and the embedding pipeline - while the other owns the Next.js application, authentication, and API integration. This gives a clean, distinct individual-contribution record for a two-person team.

11. Idea-Agnostic Requirements

Password hashing, environment-variable-based credentials, the Next.js/API integration layer, the required documentation set, Git/GitHub commit history, and the demonstration checklist all apply the same way they would to any well-built project - none of these are put at risk by this choice of domain.

12. Originality

The domain draws on real systems (e.g. CPGRAMS) for grounding, but the specific combination of semantic duplicate detection and SLA-triggered escalation as the core mechanic is not available as a ready-made tutorial or repository to copy, satisfying the originality requirement.